

Comprehensive Study Guide

CME 295: Transformers & LLMs — Lecture 2

Transformer Variants, Position Embeddings, Layer Norm, Attention Approximations, BERT

Stanford CME 295 by Afshine & Shervine Amidi · For: Applied AI/ML Engineers

Contents

1	Topic Map	2
2	Deep-Dive Notes	3
2.1	Attention Maps and Head Interpretability	3
2.2	Position Embeddings: Full Evolution	3
2.2.1	Method 1: Learned Absolute Position Embeddings	3
2.2.2	Method 2: Sinusoidal (Hardcoded) Position Embeddings	4
2.2.3	Method 3: Relative Position Bias in Attention (T5, ALiBi)	4
2.2.4	Method 4: RoPE — Rotary Position Embeddings (Modern Default)	5
2.3	Layer Normalization: Evolution	5
2.3.1	Layer Norm (Ba et al., 2016)	5
2.3.2	Post-LN vs. Pre-LN	6
2.3.3	RMSNorm (Zhang & Sennrich, 2019)	6
2.4	Attention Approximations	6
2.4.1	The $O(n^2)$ Problem	6
2.4.2	Sparse Attention: Sliding Window Attention (SWA)	7
2.4.3	Sharing Attention Heads: MQA, GQA, MHA	7
2.5	Transformer Architecture Families	7
2.5.1	T5 Family (Encoder–Decoder)	8
2.5.2	Decoder-Only Models (teaser)	8
2.6	BERT: Architecture Deep Dive	8
2.6.1	Tokenization: WordPiece	9
2.6.2	Special Tokens	9
2.6.3	Input Embedding Stack	9
2.6.4	Pre-Training: Masked Language Modeling (MLM)	9
2.6.5	Pre-Training: Next Sentence Prediction (NSP)	10
2.6.6	BERT Hyperparameters	10
2.6.7	Fine-Tuning BERT	10
2.7	BERT Variants	10
2.7.1	Knowledge Distillation	11
2.7.2	DistilBERT (Sanh et al., 2019)	11
2.7.3	RoBERTa (Liu et al., 2019)	11
3	Related Concepts You Should Also Know	12
4	Build Projects	13
5	Explain It Back	14
6	Interview Practice Questions & Answers	15
7	Self-Assessment Test	17

Topic Map

1. Attention Maps & Multi-Head Interpretability

- 1.1 Attention weight visualization; anaphora resolution example
- 1.2 What each head may specialize in

2. Position Embeddings (Evolution)

- 2.1 Learned (absolute) position embeddings — limitations
- 2.2 Hardcoded sinusoidal embeddings — dot-product similarity proof
- 2.3 Relative position bias in attention layer (T5, ALiBi)
- 2.4 RoPE (Rotary Position Embeddings) — the modern default

3. Layer Normalization (Evolution)

- 3.1 Layer Norm formula ($\mu, \sigma, \gamma, \beta$)
- 3.2 Post-LN (original Transformer) vs. Pre-LN (modern)
- 3.3 RMSNorm — simplified, fewer parameters
- 3.4 Difference from Batch Normalization

4. Attention Approximations

- 4.1 $O(n^2)$ complexity problem
- 4.2 Sparse attention: Longformer & sliding window attention (SWA)
- 4.3 Interleaving local and global attention
- 4.4 Sharing K/V heads: MQA, GQA, MHA

5. Transformer Architecture Families

- 5.1 Encoder–Decoder: T5, mT5, ByT5; span corruption objective
- 5.2 Encoder-Only: BERT, DistilBERT, RoBERTa
- 5.3 Decoder-Only: GPT series (teaser for next lectures)

6. BERT Deep Dive

- 6.1 Architecture: WordPiece tokenizer, special tokens ([CLS], [SEP], [MASK], [PAD])
- 6.2 Input embedding stack: token + position + segment
- 6.3 Pre-training: Masked Language Modeling (MLM)
- 6.4 Pre-training: Next Sentence Prediction (NSP)
- 6.5 Fine-tuning: sequence classification, token classification (QA)
- 6.6 Model variants and hyperparameter table

7. BERT Variants

- 7.1 Knowledge distillation and KL divergence
- 7.2 DistilBERT: halved layers, 97% performance
- 7.3 RoBERTa: NSP removal, dynamic masking, more data

Deep-Dive Notes

Attention Maps and Head Interpretability

Definition: Attention Map

A visualization of the attention weight matrix $A = \text{softmax}(QK^\top / \sqrt{d_k})$ for a given head and layer. Entry A_{ij} shows how much token i attends to token j . High values indicate strong relevance between those tokens.

The original “Attention Is All You Need” paper showed that different heads in the same layer learn qualitatively different attention patterns. One prominent example: in layer 5, heads 5 and 6 appear to resolve **anaphora** — for the word *its*, the model correctly attends to *law* and *application*, the two words the pronoun refers to.

What Heads Specialize In

While there are no hard constraints forcing heads to be different, empirically they diverge into roles such as:

- Syntactic relationships (subject–verb agreement)
- Coreference / anaphora resolution
- Positional patterns (attending to the next or previous token)
- Rare-word disambiguation

Each head has its own projection matrices $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$, so computations happen in parallel and are concatenated afterward.

Position Embeddings: Full Evolution

Since self-attention is permutation-equivariant, position must be injected explicitly. Four approaches have emerged, roughly chronologically.

Method 1: Learned Absolute Position Embeddings

Learned Absolute Position Embeddings

A lookup table mapping each position index $p \in \{1, \dots, L_{\max}\}$ to a trainable vector $\mathbf{e}_p \in \mathbb{R}^{d_{\text{model}}}$. These are added to token embeddings: $\mathbf{x}_p = \mathbf{t}_p + \mathbf{e}_p$. Learned end-to-end via gradient descent.

Limitations

- Can only generalize to sequence lengths seen during training (bounded by $L_{\max}$).
- Bias from training distribution: if position 2 always sees a specific type of word, the embedding encodes that bias.
- Cannot extrapolate to longer sequences at inference time.

Method 2: Sinusoidal (Hardcoded) Position Embeddings

Sinusoidal Position Embeddings (Vaswani et al., 2017)

For position m and embedding dimension index i :

$$PE_{(m, 2i)} = \sin(\omega_i \cdot m), \quad \omega_i = 10000^{-2i/d_{\text{model}}} \quad (1)$$

$$PE_{(m, 2i+1)} = \cos(\omega_i \cdot m) \quad (2)$$

No parameters to learn. The frequency ω_i is high for small i (fast oscillation) and low for large i (slow oscillation).

Why Sinusoidal? Dot-Product Similarity Proof

Define ω_i as above. For positions m and n , consider one pair of dimensions:

$$\mathbf{p}_m^{(i)} \cdot \mathbf{p}_n^{(i)} = \sin(\omega_i m) \sin(\omega_i n) + \cos(\omega_i m) \cos(\omega_i n) = \cos(\omega_i(m - n))$$

using the identity $\cos(a - b) = \cos a \cos b + \sin a \sin b$. Summing over all dimension pairs:

$$\mathbf{p}_m \cdot \mathbf{p}_n = \sum_i \cos(\omega_i(m - n))$$

This is **a function of the relative distance** ($m - n$) only. When $m = n$, $\cos(0) = 1$ is maximal. As $|m - n|$ grows, the sum decreases — matching the intuition that nearby positions should be more similar.

Benefits Over Learned

- No parameters to learn.
- Can generalize to *any* sequence length at inference time.
- Comparable empirical performance to learned embeddings.

Method 3: Relative Position Bias in Attention (T5, ALiBi)

The core insight: what we actually need is for the *attention layer* to reflect relative position, not just the inputs. Rather than adding position embeddings to token embeddings, inject position information directly into the attention score matrix.

Relative Position Bias

Modify the attention formula by adding a learnable (or deterministic) bias $b_{m,n}$ that depends on the relative distance $|m - n|$:

$$A_{mn} = \text{softmax}\left(\frac{\mathbf{q}_m \cdot \mathbf{k}_n}{\sqrt{d_k}} + b_{m,n}\right)$$

The softmax normalizes regardless, so $b_{m,n}$ effectively up- or down-weights certain position pairs.

- **T5 bias:** Bucketizes distances into learned scalars, one per head. Flexible but has learnable parameters.
- **ALiBi (Press et al., 2021):** $b_{m,n} = -\alpha_h \cdot |m - n|$ where α_h is a fixed head-specific slope. Deterministic, no extra parameters. Enables extrapolation to longer sequences than seen

during training (“train short, test long”).

Method 4: RoPE — Rotary Position Embeddings (Modern Default)

RoPE (Su et al., 2021 — RoFormer)

Instead of adding position information, **rotate** the query and key vectors by an angle proportional to their position before computing the dot product. For position m and a 2D subspace:

$$R_m = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix}$$

The rotated query and key are $\tilde{\mathbf{q}}_m = R_m \mathbf{q}_m$ and $\tilde{\mathbf{k}}_n = R_n \mathbf{k}_n$.

Why Rotation Gives Relative Distance

$$\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n = (R_m \mathbf{q}_m)^\top (R_n \mathbf{k}_n) = \mathbf{q}_m^\top R_m^\top R_n \mathbf{k}_n = \mathbf{q}_m^\top R_{n-m} \mathbf{k}_n \quad (3)$$

using $R_m^\top R_n = R_{n-m}$ (rotation matrices compose by angle subtraction). The dot product is a function of $\mathbf{q}_m$, $\mathbf{k}_n$, and **only the relative distance** ($n - m$), not absolute positions.

- For $d > 2$: apply 2D rotation block-wise to consecutive pairs of dimensions, with different θ per block (analogous to sinusoidal ω_i).
- **Long-term decay**: the upper bound of $\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n$ decays as $|m - n|$ grows, producing the desired behavior that distant tokens contribute less.
- Used by: LLaMA, Mistral, Falcon, Qwen, and most modern open-source LLMs.

Table 1: Position Embedding Method Comparison

Method	Learnable	Extrapolates	Where Applied	Used By
Learned absolute	Yes	No	Input	Original BERT
Sinusoidal	No	Yes	Input	Original Transformer
T5 bias	Yes	Partial	Attention	T5 family
ALiBi	No	Yes	Attention	Bloom
RoPE	No	Yes	Attention (Q,K)	LLaMA, Mistral

Layer Normalization: Evolution

Layer Norm (Ba et al., 2016)

Layer Normalization Formula

Given an activation vector $\mathbf{x} \in \mathbb{R}^d$:

$$\mu = \frac{1}{d} \sum_{j=1}^d x_j, \quad \sigma^2 = \frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2$$

$$\hat{x}_j = \frac{x_j - \mu}{\sigma + \epsilon}, \quad \text{LN}(\mathbf{x})_j = \gamma_j \hat{x}_j + \beta_j$$

where $\gamma, \beta \in \mathbb{R}^d$ are **learned** scale and shift parameters. ϵ is a small constant for numerical stability.

Motivation: During training, activations can have extreme values in some dimensions and near-zero in others. Normalizing within each layer reduces **internal covariate shift**, improving gradient flow and convergence speed.

Layer Norm vs. Batch Norm

- **Batch Norm:** Normalizes across the *batch dimension* for each feature. Requires sufficiently large batches; statistics differ between train and inference.
- **Layer Norm:** Normalizes across the *feature dimension* for each sample independently. No batch dependency. Preferred for NLP and Transformers.

Post-LN vs. Pre-LN

- **Post-LN (original Transformer):** $\text{LN}(\mathbf{x} + \text{sublayer}(\mathbf{x}))$ — normalize *after* the residual addition.
- **Pre-LN (modern default):** $\mathbf{x} + \text{sublayer}(\text{LN}(\mathbf{x}))$ — normalize *before* the sublayer. The gradient flows through the residual stream without passing through a normalization operation, improving training stability in deep networks.

RMSNorm (Zhang & Sennrich, 2019)

Root Mean Square Normalization

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2}, \quad \text{RMSNorm}(\mathbf{x})_j = \frac{x_j}{\text{RMS}(\mathbf{x})} \cdot \gamma_j$$

Drops the mean subtraction and the β term. Learns only γ .

- **Why:** The authors show that re-centering (subtracting mean) is less important than re-scaling. Removing it reduces parameters and is computationally cheaper.
- Comparable convergence to Layer Norm.
- Used by: LLaMA, Mistral, most modern LLMs.

Attention Approximations

The $O(n^2)$ Problem

The core bottleneck of standard self-attention: computing $QK^\top$ requires $O(n^2 \cdot d)$ time and $O(n^2)$ memory. For $n = 100,000$ tokens, the attention matrix alone is $100,000 \times 100,000$ — intractable.

Sparse Attention: Sliding Window Attention (SWA)

Sliding Window Attention

Each token attends only to its w nearest neighbors (a local window of radius $w/2$ on each side). Complexity drops from $O(n^2)$ to $O(n \cdot w)$.

Effective Receptive Field

Even with a window of size w per layer, after k stacked layers, a token has an effective receptive field of $k \cdot w$ tokens — analogous to receptive field expansion in CNNs via stacked convolutions. Mistral 7B uses SWA: window = 4096 tokens per layer but n stacked layers means the full context is accessible through the layer stack.

Global + Local Interleaving

In practice, models interleave layers with:

- **Local (SWA) layers:** attend to nearby tokens only ($O(nw)$)
- **Global layers:** attend to all tokens ($O(n^2)$, but fewer of these layers)

Certain tokens (e.g., [CLS]) may always receive global attention to act as information aggregation points (Longformer approach).

Sharing Attention Heads: MQA, GQA, MHA

Motivation: During autoregressive decoding, keys and values from all past tokens must be recomputed or cached (the **KV cache**). With h heads, the KV cache grows as $O(n \cdot h \cdot d_k)$, which becomes very large for long contexts.

MHA, GQA, MQA — Formally

Let h = number of query heads, g = number of KV groups.

- **Multi-Head Attention (MHA):** $g = h$. Every head has its own $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$. Maximum expressivity, maximum KV cache.
- **Group Query Attention (GQA):** $g < h$. h/g query heads share one K/V projection. KV cache reduced by factor h/g . Used in LLaMA 3, Mistral.
- **Multi-Query Attention (MQA):** $g = 1$. All query heads share a single K/V projection. Maximum KV cache saving, some quality loss.

Why Share K/V but not Q?

The query represents “what this token is looking for” — diversity here is semantically meaningful (each head can ask a different question). Keys and values represent “what am I and what do I contain” — sharing them reduces KV cache pressure without significantly harming representation quality. Empirically, GQA closely matches MHA performance at reduced memory cost.

Transformer Architecture Families

Table 2: Three Families of Transformer-Based Models

Family	Architecture	Training Objective	Examples
Encoder–Decoder	Full Transformer	Span corruption (T5), next-token (original)	T5, mT5, ByT5
Encoder-Only	Encoder only	MLM + NSP (BERT)	BERT, DistilBERT, RoBERTa
Decoder-Only	Decoder only (masked self-attention + FFN)	Next-token prediction	GPT series, LLaMA, Mistral

T5 Family (Encoder–Decoder)

Span Corruption Objective (T5)

Instead of next-token prediction, T5 trains on **span corruption**: randomly mask contiguous spans of tokens in the input, replace each span with a unique sentinel token (e.g., `<extra_id_0>`), and train the decoder to reconstruct the masked spans in sequence.

Example:

Input to encoder: My `<extra_id_0>` is reading `<extra_id_1>`.

Decoder target: `<extra_id_0>` teddy bear `<extra_id_1>` quietly

T5 variants:

- **mT5**: Multilingual T5, trained on 101 languages.
- **ByT5**: Byte-level T5, no tokenizer. Vocabulary = 256 (all byte values). More robust to typos and rare languages; longer sequences.

Decoder-Only Models (teaser)

Remove the encoder entirely. Each decoder block has only:

1. Masked self-attention (causal)
2. FFN

No cross-attention (nothing to cross-attend to). Trained on next-token prediction. Scales extremely well. Dominant paradigm since 2020. Will be covered in depth in future lectures.

BERT: Architecture Deep Dive

BERT = Bidirectional Encoder Representations from Transformers (Devlin et al., 2018)

BERT takes the encoder stack from the Transformer and pre-trains it with two self-supervised proxy tasks (MLM + NSP) on large unlabeled corpora. It produces rich, **bidirectional** contextual representations suitable for fine-tuning on classification tasks.

Bidirectionality: Each token attends to all other tokens (no causal mask). Contrast with GPT’s masked (causal) self-attention where each token only sees tokens before it. BERT’s full mutual attention is the defining property.

Tokenization: WordPiece

WordPiece Algorithm

A learned subword tokenizer that iteratively merges token pairs by maximizing the **likelihood** of the training corpus under the tokenized representation (as opposed to BPE which maximizes frequency count). Produces a vocabulary of $\sim 30\text{K}$ tokens. Continuation pieces are prefixed with **##**.

Special Tokens

- **[CLS]**: Classification token. Prepended to every input. After passing through all encoder layers, the output embedding of [CLS] serves as a sequence-level representation for downstream classification.
- **[SEP]**: Separator token. Marks the boundary between two input segments (sentences A and B) and appears at the end of each.
- **[MASK]**: Masking token. Replaces selected tokens during MLM pre-training.
- **[PAD]**: Padding token. Used to bring sequences in a batch to uniform length. Typically masked out in attention.

Input Embedding Stack

BERT's input representation is a sum of three learned embeddings:

BERT Input Representation

$$\mathbf{x}_i = \underbrace{\mathbf{e}_{\text{token}}(t_i)}_{\text{Token embedding}} + \underbrace{\mathbf{e}_{\text{pos}}(i)}_{\text{Position embedding}} + \underbrace{\mathbf{e}_{\text{seg}}(s_i)}_{\text{Segment embedding}}$$

where $s_i \in \{A, B\}$ denotes which sentence the token belongs to.

Segment embedding is new to BERT: a shared embedding $\mathbf{e}_A$ for all tokens in sentence A and $\mathbf{e}_B$ for all tokens in sentence B. Designed to help the NSP task by marking sentence boundaries in the embedding space. (RoBERTa later showed it contributes little and removed it.)

Pre-Training: Masked Language Modeling (MLM)

MLM Protocol

From the input, 15% of tokens are selected for prediction. Of those selected:

- **80%**: Replace with [MASK] token.
- **10%**: Replace with a random vocabulary token.
- **10%**: Keep the original token unchanged.

The model must predict the original token for every selected position.

Why Not Just Mask 100%?

If all selected tokens were replaced with [MASK], the model would learn that [MASK] always needs predicting — creating a train/inference mismatch since [MASK] never appears at inference time. The 10% random + 10% unchanged acts as regularization, forcing the model to maintain good representations for *all* tokens, not just masked ones.

The MLM task forces the model to use **bidirectional context**: to predict the masked word, it must look at both left and right neighbors. This is why BERT’s representations are bidirectional.

Pre-Training: Next Sentence Prediction (NSP)

NSP Protocol

Sample two sentences A and B:

- **50% of the time:** B is the actual next sentence following A in the corpus (label: IsNext).
- **50% of the time:** B is a random sentence from the corpus (label: NotNext).

Input: [CLS] A [SEP] B [SEP]. The [CLS] output is projected to a binary classifier.

Note: RoBERTa (2019) showed that removing NSP has negligible impact or even slightly improves performance. The hypothesis is that NSP is too easy — random sentences are trivially different from consecutive ones.

BERT Hyperparameters

Table 3: BERT Model Configurations

Model	L (layers)	H (dim)	A (heads)	Params
BERT-Tiny	2	128	2	4M
BERT-Mini	4	256	4	11M
BERT-Small	4	512	8	30M
BERT-Medium	8	512	8	42M
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M

Fine-Tuning BERT

Fine-Tuning Strategy

After pre-training, add task-specific heads on top of the frozen (or partially frozen) BERT encoder and train on labeled data:

- **Sequence classification** (e.g., sentiment): project [CLS] output through a linear layer to class logits. Requires very little labeled data.
- **Token classification** (e.g., NER, QA span detection): project each token’s output embedding through a linear layer. For QA: two separate classifiers predict start and end span positions.

Freezing Strategy

Common practice: freeze the first few encoder layers (they learn generic representations) and fine-tune only the top layers and the task head. The closer the downstream task is to the pre-training distribution, the fewer layers need fine-tuning.

BERT Variants

Knowledge Distillation

Knowledge Distillation (Hinton et al., 2014)

Train a smaller **student** model to match the **output distribution** (soft targets) of a larger **teacher** model, rather than training on hard one-hot labels.

Objective: minimize the KL divergence between student and teacher distributions:

$$\mathcal{L}_{\text{distill}} = D_{\text{KL}}(P_T \| P_S) = \sum_v P_T(v) \log \frac{P_T(v)}{P_S(v)}$$

Note: if P_T is a hard label (one-hot), this reduces to the standard cross-entropy loss.

Why Soft Targets Are Better

The teacher’s full probability distribution over the vocabulary encodes *similarity information* that hard labels do not. For example, if the teacher assigns 0.01 probability to “cat” and 0.001 to “dog” when the correct answer is “animal,” the student learns that “cat” is more similar to “animal” than “dog” is. This “dark knowledge” is lost in a one-hot label.

DistilBERT (Sanh et al., 2019)

- Reduces layers from 12 to 6 (BERT-Base to DistilBERT-Base).
- Uses distillation to transfer knowledge from BERT-Base (teacher) to DistilBERT (student).
- Also initializes student weights from every other layer of the teacher.
- **Result:** $\sim 1.6\times$ faster, $\sim 40\%$ fewer parameters, retains $\sim 97\%$ of BERT-Base performance on GLUE.

RoBERTa (Liu et al., 2019)

RoBERTa (Robustly Optimized BERT Pre-training Approach) is an empirical study of what actually matters in BERT pre-training.

Table 4: RoBERTa Changes vs. BERT

Change	BERT	RoBERTa
NSP objective	Used	Removed (no impact)
Segment encoding	Used	Removed
Masking	Static (fixed at preprocessing)	Dynamic (re-masked each epoch)
Pretraining data	16 GB (BookCorpus + Wikipedia)	160 GB (+ CC-News, OpenWebText, Stories)
Training steps	1M steps, batch size 256	500k steps, batch size 8k

Result: +4% on downstream benchmarks with the same architecture. The dominant lesson: BERT was massively undertrained. More data and longer training matter more than architectural tricks like NSP.

Related Concepts You Should Also Know

1. **KV Cache:** During autoregressive decoding, the key and value matrices for all previously generated tokens are stored (cached) to avoid recomputation. Cache size = $O(n \cdot h \cdot d_k)$ per layer. GQA/MQA directly reduce this by sharing K/V heads.
2. **FlashAttention (Dao et al., 2022):** A hardware-aware exact attention algorithm that avoids materializing the full $n \times n$ attention matrix in HBM (GPU memory) by computing attention in tiles. Achieves $O(n)$ memory and 2–4× wall-clock speedup with identical numerical output to standard attention.
3. **Internal Covariate Shift:** The change in input distribution to a layer as parameters of preceding layers update. Layer Norm and Batch Norm both combat this, but via different normalization axes.
4. **Residual Connection (Skip Connection):** $\mathbf{x}' = \mathbf{x} + f(\mathbf{x})$. Allows gradients to flow directly from later to earlier layers without passing through nonlinearities. Critical for training deep Transformers. The “Add” in “Add & Norm.”
5. **GLUE / SuperGLUE Benchmarks:** Standard NLP benchmark suites used to evaluate BERT-style models. GLUE: 9 classification tasks (SST-2 for sentiment, MNLI for entailment, etc.). SuperGLUE: harder follow-up. BERT, RoBERTa, and their variants are commonly compared on these.
6. **ELMo (Peters et al., 2018):** Embeddings from Language Models. Used a bidirectional LSTM (not Transformer) to produce contextual word representations. Published the same year as BERT; outclassed by BERT due to LSTM’s scaling limitations.
7. **Teacher Forcing:** During decoder training, the ground-truth previous token is fed as input (not the model’s own prediction). Speeds up training but creates a train/inference mismatch (exposure bias).
8. **ALBERT:** Another BERT variant (not covered in lecture). Uses factorized embedding parameterization (separates vocabulary embedding size from hidden size) and cross-layer parameter sharing to reduce parameters.
9. **Anaphora Resolution:** The NLP task of linking pronouns or noun phrases to the entities they refer to (e.g., “The lawyer said *its* application was ready” → *its* refers to *lawyer*’s entity). One of the phenomena interpretably captured by multi-head attention.
10. **Context Window Limits:** Early BERT was limited to 512 tokens. Longformer (with SWA) extended this to 4096+. Modern LLMs (LLaMA 3, Claude, GPT-4) support 128K+ tokens, enabled by RoPE, GQA, and FlashAttention together.

Build Projects

1. **RoPE from Scratch** (*3–5 hours, high signal*)

Implement RoPE in PyTorch. Start with the 2D case (prove the relative-distance property by computing $\tilde{q}_m^\top \tilde{k}_n$ symbolically). Then implement the block-diagonal extension for arbitrary d . Plug it into a minimal self-attention module and verify that the attention score matrix is truly a function of $(m - n)$ only by checking symmetry. Write a short blog post or notebook explaining the math. This is a high-signal interview piece for LLM engineering roles.

2. **BERT Fine-Tuning Pipeline** (*4–6 hours, very high signal*)

Fine-tune BERT-Base (or DistilBERT for speed) on a real classification dataset using HuggingFace Transformers. Experiment with: (a) freezing all layers except the classification head vs. full fine-tuning, (b) varying learning rate with warmup schedule, (c) comparing BERT-Base vs. DistilBERT performance and latency. Document the trade-offs. This directly maps to industry use cases.

3. **Attention Complexity Profiler** (*3–4 hours, medium signal*)

Write a PyTorch script that profiles standard MHA vs. a windowed SWA implementation for increasing sequence lengths ($n = 512, 1024, 2048, 4096, 8192$). Measure: peak GPU memory, forward-pass time, and output similarity (MHA vs. SWA). Plot the results. Demonstrates understanding of the $O(n^2)$ bottleneck and practical mitigation.

Explain It Back

1. You've been asked to choose a positional encoding method for a new LLM that needs to handle documents up to 200K tokens, when the training set only has documents up to 8K tokens. Walk through the trade-offs of sinusoidal, ALiBi, and RoPE, and justify your choice.
2. A team member argues: "Let's keep using Post-LN like in the original Transformer — it worked in the paper." Explain (a) what Post-LN vs. Pre-LN means, (b) why Pre-LN is preferred for deep networks, and (c) why RMSNorm is an improvement over standard Layer Norm.
3. A PM asks: "We already have BERT working. Why would we want DistilBERT?" Give a concrete business case (latency, cost, performance trade-off) and explain how distillation avoids training from scratch.
4. Explain why BERT is "bidirectional" while GPT is not, using a specific example sentence. Then explain what this means for the types of tasks each model is suited for.

Interview Practice Questions & Answers

L1 — What is RoPE and why do most modern LLMs use it?

Answer: RoPE (Rotary Position Embeddings) encodes position by rotating query and key vectors in the attention layer by an angle proportional to their position. The key property is that the dot product $\tilde{q}_m^\top \tilde{k}_n$ depends only on the relative distance $(n - m)$, not absolute positions. This is mathematically guaranteed by the composition of rotation matrices: $R_m^\top R_n = R_{n-m}$. Benefits over alternatives: no learned parameters (unlike T5 bias), no extrapolation bound (unlike learned absolute embeddings), and natural long-term decay as distance grows. Used by LLaMA, Mistral, Falcon, and virtually all modern open-source LLMs.

L1 — What is the difference between GQA and MQA? When would you use each?

Answer: In MHA (standard), every attention head has its own W_Q, W_K, W_V projections. In MQA (Multi-Query Attention), all query heads share one single W_K, W_V pair, minimizing KV cache memory. In GQA (Grouped Query Attention), query heads are divided into g groups, each group sharing one W_K, W_V pair. GQA is a soft middle ground between MHA expressivity and MQA efficiency. Use MQA for extreme memory-constrained serving (maximum KV cache reduction, some quality loss). Use GQA when you want near-MHA quality with significantly reduced cache (e.g., LLaMA 3 uses GQA). Use MHA when you have sufficient memory and want maximum expressivity.

L2 — Why did RoBERTa remove the NSP objective from BERT, and what did they learn?

Answer: RoBERTa removed NSP and found essentially no drop in performance (and sometimes slight improvements). The hypothesis is that NSP is too easy — the negative examples are random sentences from completely different documents, making them trivially distinguishable from true consecutive sentences via topic alone, without requiring deeper discourse understanding. The model was learning a trivial shortcut, not meaningful sentence relationships. The broader lesson from RoBERTa: BERT was dramatically undertrained. Scaling data from 16 GB to 160 GB, using dynamic masking (re-mask each epoch), and larger batch sizes gave +4% on downstream benchmarks with the *same* architecture.

L2 — Explain the MLM masking strategy (80/10/10) and why each component exists.

Answer: Of the 15% of tokens selected for prediction: 80% are replaced with [MASK], 10% with a random word, and 10% are kept unchanged. The 80% [MASK] is the primary training signal — the model must predict the original from context. The 10% random replacement prevents the model from learning “if it’s [MASK], just predict something” — it must maintain good representations for all tokens since any could be the prediction target. The 10% unchanged forces the model to not assume every token needs correction — it must also correctly represent tokens that are already in their correct form. Together they create a more robust representation that works at inference time when no [MASK] tokens appear.

L2 — What is the KV cache and how does GQA reduce its memory footprint?

Answer: During autoregressive decoding, computing attention for a new token requires the keys and values of all previous tokens. Rather than recomputing these at every step, they are cached in GPU memory. For a model with h heads, L layers, n generated tokens, and d_k per head, the KV cache size is $2 \cdot L \cdot h \cdot n \cdot d_k$ (factor 2 for K and V). With GQA using g KV groups (instead of h), this becomes $2 \cdot L \cdot g \cdot n \cdot d_k$, reducing memory by a factor of h/g . For LLaMA 3-70B with $h = 64$ heads and $g = 8$ groups, the KV cache is $8\times$ smaller than MHA, enabling longer context windows within the same memory budget.

L3 — Why is Pre-LN preferred over Post-LN in modern deep Transformers?

Answer: In Post-LN, the gradient path from the loss to the input of an early layer always passes through layer normalization operations. At initialization, this can make the gradient signal to early layers very small, requiring careful learning rate warmup schedules. In Pre-LN, the residual stream $\mathbf{x}$ always has a direct gradient path to the loss (via the residual connections), regardless of depth. The sublayer's input is normalized, but the residual bypass is clean. This makes Pre-LN models easier to train without extensive warmup and enables stable training of very deep networks ($N > 24$ layers) that Post-LN struggles with.

L3 — Prove that sinusoidal dot products are a function of relative distance only.

Answer: For a single dimension pair i , the position embedding components are $\sin(\omega_i m)$ and $\cos(\omega_i m)$ for position m . The contribution of this pair to the dot product between position embeddings $\mathbf{p}_m$ and $\mathbf{p}_n$ is:

$$\sin(\omega_i m) \sin(\omega_i n) + \cos(\omega_i m) \cos(\omega_i n) = \cos(\omega_i m - \omega_i n) = \cos(\omega_i (m - n))$$

using the identity $\cos(a - b) = \cos a \cos b + \sin a \sin b$. Summing over all $d/2$ dimension pairs: $\mathbf{p}_m \cdot \mathbf{p}_n = \sum_{i=0}^{d/2-1} \cos(\omega_i (m - n))$, which depends only on the relative distance $(m - n)$, not on m or n individually.

L1 — What is the [CLS

token in BERT and how does it work?] **Answer:** [CLS] is a special classification token prepended to every BERT input. It has no semantic meaning by itself — it is a placeholder. After the input passes through all L encoder layers, every token's representation has attended to every other token via the bidirectional self-attention. The [CLS] output embedding therefore contains a mixture of all input token representations, weighted by what the model learned is relevant for prediction. A linear layer is placed on top of this [CLS] output to produce class logits. The [CLS] representation is sequence-level because it was explicitly trained (via NSP and downstream fine-tuning) to be useful for whole-sequence classification tasks.

Self-Assessment Test

Scoring: 16–20 = solid mastery; 12–15 = revisit weak areas; <12 = re-study before moving on.

Multiple Choice (1 point each)

Q1. RoPE encodes position by:

(a) Adding sinusoidal vectors to token embeddings (b) Rotating Q and K vectors in the attention layer (c) Learning a bias per position per head (d) Appending position IDs to the vocabulary

Q2. In GQA with $h = 32$ query heads and $g = 4$ groups, how many K/V projection matrices are there?

(a) 32 (b) 4 (c) 8 (d) 128

Q3. RMSNorm differs from Layer Norm by:

(a) Normalizing across the batch instead of features (b) Removing the mean subtraction and the β term (c) Using ℓ_2 norm instead of standard deviation (d) Applying normalization after the residual connection

Q4. In BERT's MLM, if a token is selected for prediction, it is replaced with [MASK]:

(a) 100% of the time (b) 80% of the time (c) 15% of the time (d) 10% of the time

Q5. Which property of the sinusoidal positional encoding is its main advantage over learned embeddings?

(a) Lower perplexity on downstream tasks (b) Can generalize to sequence lengths not seen in training (c) Requires fewer parameters (d) Both (b) and (c)

Q6. BERT's NSP objective was shown by RoBERTa to:

(a) Dramatically improve NER performance (b) Have negligible impact on performance (c) Be essential for question answering (d) Require segment embeddings to work

Q7. In the Longformer's sliding window attention, what is the relationship to convolutional receptive fields?

(a) SWA is mathematically equivalent to a 1D convolution (b) Stacking SWA layers expands the effective receptive field analogously to stacked convolutions (c) SWA uses convolutional kernels instead of attention (d) There is no relationship

Q8. The KV cache is relevant because:

(a) It stores all training data for fast lookup (b) During inference, K and V of past tokens are cached to avoid recomputation at each generation step (c) It caches gradient values for faster backprop (d) It replaces the need for positional encoding

Short Answer (1.5 points each)

Q9. Write the RMSNorm formula and explain in one sentence why it removes the mean subtraction.

Q10. What are the three components summed in BERT's input embedding for each token? Name all three and explain what is new compared to the original Transformer.

Q11. In Pre-LN, write the formula for passing an input $\mathbf{x}$ through one Transformer sublayer (e.g., self-attention). How does this differ from Post-LN?

Q12. What is the distillation loss (KL divergence) minimizing? When does it reduce to standard cross-entropy?

Q13. Why are K/V heads shared across query groups in GQA, but not Q heads?

Q14. State two specific changes RoBERTa made to the BERT pre-training process and the result.

Scenario (2 points)

Q15. Your team needs to deploy a sentiment classifier for customer reviews in a latency-sensitive mobile application. You have 50K labeled examples. The constraint: inference must be $< 20\text{ms}$ on a mobile CPU. Compare BERT-Base, DistilBERT, and a fine-tuned RoBERTa. Which would you choose, why, and what fine-tuning strategy would you use?

Answer Key

Q1. (b) — Rotating Q and K in the attention layer.

Q2. (b) — 4. There are $g = 4$ KV groups, so 4 distinct W_K and W_V matrices.

Q3. (b) — Removes mean subtraction and the β shift parameter; normalizes by RMS only.

Q4. (b) — 80%. The other 10% get a random token, and 10% remain unchanged.

Q5. (d) — Both (b) and (c). No learned parameters, and extrapolates to unseen lengths.

Q6. (b) — NSP had negligible impact; RoBERTa removed it without performance loss.

Q7. (b) — Stacking SWA layers creates an expanding effective receptive field, analogous to CNNs.

Q8. (b) — Caches K and V for previously generated tokens to avoid recomputing them at each decoding step.

Q9.

$$\text{RMSNorm}(\mathbf{x})_j = \frac{x_j}{\sqrt{\frac{1}{d} \sum_k x_k^2}} \cdot \gamma_j$$

The mean subtraction is dropped because the authors showed that re-centering contributes little to training stability compared to re-scaling, and removing it reduces computation and parameters.

Q10. Token embedding + Position embedding + **Segment embedding**. The segment embedding is new to BERT — a shared learned embedding ($\mathbf{e}_A$ or $\mathbf{e}_B$) added to all tokens in the same sentence segment to support the NSP task. The original Transformer had no segment embedding.

Q11. Pre-LN: $\mathbf{x}' = \mathbf{x} + \text{sublayer}(\text{LN}(\mathbf{x}))$. Post-LN: $\mathbf{x}' = \text{LN}(\mathbf{x} + \text{sublayer}(\mathbf{x}))$. In Pre-LN, gradients flow directly through the residual $\mathbf{x}$ without passing through LN, enabling more stable training of deep networks.

Q12. KL divergence minimizes $D_{\text{KL}}(P_T \| P_S) = \sum_v P_T(v) \log \frac{P_T(v)}{P_S(v)}$, which measures how poorly the student distribution P_S approximates the teacher's world view P_T . When P_T is a one-hot hard label, $P_T(v) = 1$ for the correct class and 0 elsewhere, reducing the KL divergence to $-\log P_S(\text{correct}) = \text{cross-entropy}$.

Q13. Query heads remain diverse because different queries (“what am I looking for?”) capture different types of relationships. Sharing K/V heads reduces KV cache memory proportionally to h/g while having minimal quality impact — empirically, the keys and values (“what I contain and provide”) are more homogeneous across heads than the queries.

Q14. Any two of: (1) Removed NSP objective $\rightarrow$ no performance loss. (2) Dynamic masking (re-mask each epoch) instead of static $\rightarrow$ more diverse training signal. (3) Data: 16 GB $\rightarrow$ 160 GB $\rightarrow$ +4% on benchmarks. (4) Larger batch size (8k vs. 256 with more steps).

Q15. Choose DistilBERT. BERT-Base (110M params) is far too slow for 20ms on mobile CPU — typical BERT-Base inference on CPU is 100–200ms. RoBERTa has the same architecture as BERT-Base (340M for Large), so same latency issue. DistilBERT-Base (66M params, 6 layers) is $\sim 1.6\times$ faster than BERT-Base and retains 97% performance, making it viable for mobile. Fine-tuning strategy: (1) Load pre-trained DistilBERT from HuggingFace. (2) Add a linear classification head on top of the [CLS] output. (3) With 50K labeled examples, full fine-tuning (unfreeze all layers) with a low learning rate (2×10^{-5}) and linear warmup over 10%

of steps. (4) If latency still exceeds 20ms, quantize to INT8 (post-training quantization) for an additional $\sim 2\times$ speedup with minimal accuracy loss.

End of Study Guide — Lecture 2

Source: Stanford CME 295, Fall 2025. Afshine & Shervine Amidi.